

Álgebra Relacional



Eduardo Ogasawara

eogasawara@cefet-rj.br | <https://eic.cefet-rj.br/~eogasawara>



O que é Álgebra Relacional?

Definição e Propósito

A **álgebra relacional** é um conjunto básico e fundamental de operações matemáticas projetadas especificamente para o modelo relacional de banco de dados. Essas operações formam a base teórica que sustenta todas as consultas realizadas em bancos de dados relacionais modernos.

Uma **expressão da álgebra relacional** consiste em uma sequência organizada dessas operações, permitindo a manipulação e recuperação de dados de forma estruturada e eficiente.

Objetivo Principal

O principal objetivo da álgebra relacional é fornecer uma base matemática sólida que permite o cálculo e a execução de consultas declaradas em linguagens de nível mais alto, como SQL.

Ela serve como a **ponte conceitual** entre como os usuários expressam suas necessidades de dados e como o sistema de gerenciamento de banco de dados (SGBD) efetivamente recupera essas informações.

Álgebra Relacional como Linguagem Procedimental

Natureza Procedimental

A álgebra relacional é uma **linguagem procedimental** que especifica *como* obter os dados desejados através de uma sequência explícita de passos.

Sequência Ordenada

Uma consulta é definida por uma **sequência ordenada de operações**, onde a ordem importa e cada passo depende do resultado anterior.

Operações e Resultados

Cada operação recebe **uma ou duas relações** como entrada e produz **uma nova relação** como saída, garantindo o fechamento algébrico.

Composição

O resultado final depende da **composição cuidadosa das operações**, onde a saída de uma alimenta a entrada da próxima.

📌 **Importante:** Diferentemente de linguagens declarativas como SQL, onde você especifica *o que* deseja, a álgebra relacional exige que você especifique *como* obter o resultado.

Esquema de Exemplo Usado

Para ilustrar as operações da álgebra relacional, utilizaremos um esquema de banco de dados bancário simplificado.

agencia

nomeAgencia, cidadeAgencia, ativo — armazena informações sobre as agências bancárias.

conta

numeroConta, nomeAgencia*, saldo — registra as contas bancárias e seus saldos.

depositante

nomeCliente*, numeroConta* — relaciona clientes às suas contas bancárias.

cliente

nomeCliente, ruaCliente, cidadeCliente — contém os dados cadastrais dos clientes do banco.

emprestimo

numeroEmprestimo, nomeAgencia*, quantia — documenta os empréstimos concedidos pelo banco.

tomador

nomeCliente*, numeroEmprestimo* — relaciona clientes aos seus empréstimos.



Nota: Os atributos marcados com asterisco (*) são **chaves estrangeiras**, estabelecendo relacionamentos entre as tabelas do esquema.

Operação Seleção (σ)

A operação de **seleção**, denotada pelo símbolo grego σ (**sigma**), é uma operação unária fundamental que permite filtrar tuplas (linhas) de uma relação com base em um predicado (condição) específico.

A seleção examina cada tupla da relação de entrada e inclui no resultado apenas aquelas tuplas que **satisfazem a condição especificada**. É análoga à cláusula WHERE em SQL.

Notação Formal

$$\sigma_{\text{predicado}}(R)$$

Onde *predicado* é a condição de filtragem e *R* é a relação de entrada.

Atributos Preservados

Mantém todos os atributos (colunas) da relação original.

Redução de Tuplas

Reduz o número de tuplas, retornando apenas as que satisfazem o predicado.

Operadores de Comparação

O predicado pode usar $=$, $\neq$, $<$, $>$, $\leq$, $\geq$.

Operadores Lógicos

Condições podem ser combinadas com $\wedge$ (e), $\vee$ (ou) e $\neg$ (não).

Operação Seleção – Exemplo Prático

Demonstraremos como a operação de seleção funciona utilizando exemplos concretos do nosso esquema bancário. A seleção é uma operação unária que permite **filtrar tuplas (linhas)** de uma **relação** com base em uma condição específica, mantendo todas as colunas.

Relação R (conta) - Entrada:

numeroConta	nomeAgencia	saldo
A-101	Centro	5000
A-102	Paulista	12000
A-103	Centro	8500
A-104	Jardins	3200

Exemplo de Seleção

$\sigma_{\text{saldo} > 5000}(\text{conta})$

Resultado: Retorna apenas as tuplas (linhas) onde o saldo da conta é maior que 5000. Todas as colunas são mantidas.

numeroConta	nomeAgencia	saldo
A-102	Paulista	12000
A-103	Centro	8500

 A operação de seleção filtra linhas (subconjunto horizontal), mantendo todas as colunas.

Operação Projeção (π)

A operação de **projeção**, representada pela letra grega π , é outra operação unária essencial que permite selecionar atributos (colunas) específicos de uma relação, descartando os demais.

Diferentemente da seleção, que filtra linhas, a projeção **filtra colunas**, retornando apenas os atributos de interesse. Corresponde à cláusula SELECT DISTINCT em SQL: como relações são conjuntos, a projeção elimina duplicatas, enquanto um SELECT simples as mantém.

Notação Formal

$$\pi_{\text{atributo1, atributo2, ...}}(R)$$

Onde a lista de atributos especifica quais colunas manter.

Redução de Atributos

Reduz o número de colunas no resultado, mantendo apenas os atributos especificados.

Ordem dos Atributos

A ordem das colunas no resultado corresponde à ordem especificada na operação.

Eliminação de Duplicatas

Mantém todas as tuplas, mas elimina duplicatas automaticamente no resultado.

Combinação com Outras Operações

Pode ser combinada com seleção e outras operações para consultas mais complexas.

Operação Projeção – Exemplo Prático

Demonstraremos como a projeção funciona utilizando exemplos concretos do nosso esquema bancário.

Relação R (conta) - Entrada:

numeroConta	nomeAgencia	saldo
A-101	Centro	5000
A-102	Paulista	12000
A-103	Centro	8500
A-104	Jardins	3200

Exemplo 1

$\pi_{\text{nomeAgencia, saldo}}(\text{conta})$

Resultado: Retorna apenas as colunas nomeAgencia e saldo, eliminando numeroConta.

nomeAgencia	saldo
Centro	5000
Paulista	12000
Centro	8500
Jardins	3200

Exemplo 2

$\pi_{\text{nomeAgencia}}(\text{conta})$

Resultado: Retorna apenas agências únicas (duplicatas removidas).

nomeAgencia
Centro
Paulista
Jardins

📌 Note que "Centro" aparece apenas uma vez, apesar de existir em duas tuplas originais.

Operação União ($\cup$)

A operação de **união**, denotada pelo símbolo $\cup$, é uma operação binária que combina duas relações compatíveis, produzindo uma nova relação contendo todas as tuplas que aparecem em **pelo menos uma** das relações originais.

Compatibilidade de União

Para unir duas relações R e S , elas devem ter o mesmo número de atributos com domínios compatíveis.

Eliminação de Duplicatas

A união elimina automaticamente tuplas duplicadas — se uma tupla aparece em ambas as relações, ela aparece apenas uma vez em $R \cup S$.

Propriedades Matemáticas

A união é **comutativa** ($R \cup S = S \cup R$) e **associativa** ($(R \cup S) \cup T = R \cup (S \cup T)$).

Notação Formal: $R \cup S$ resulta em uma relação contendo todas as tuplas de R , mais todas as tuplas de S , sem duplicatas.

Operação União – Exemplo Prático

Vamos ilustrar a operação de união com um exemplo concreto usando duas relações compatíveis.

Relação R

Nome	Cidade
João	São Paulo
Maria	Rio de Janeiro
Pedro	Belo Horizonte

Clientes da região Sudeste

Relação S

Nome	Cidade
Maria	Rio de Janeiro
Ana	Curitiba
Carlos	Porto Alegre

Clientes da região Sul

$R \cup S$

Nome	Cidade
João	São Paulo
Maria	Rio de Janeiro
Pedro	Belo Horizonte
Ana	Curitiba
Carlos	Porto Alegre

Todos os clientes únicos

📄 **Observação:** A tupla (Maria, Rio de Janeiro) aparecia em ambas as relações, mas aparece apenas **uma vez** no resultado da união. O resultado contém 5 tuplas, não 6.

Operação de Interseção ($\cap$)

A operação de **interseção**, representada pelo símbolo $\cap$, é uma operação binária que produz uma nova relação contendo apenas as tuplas que aparecem **simultaneamente em ambas as relações** de entrada.

Assim como a união, a interseção requer que as relações sejam **compatíveis em união**, com o mesmo número de atributos e domínios correspondentes compatíveis.

Notação Formal

$$R \cap S$$

Retorna todas as tuplas que existem tanto em R quanto em S.

Propriedades Essenciais

A interseção é comutativa ($R \cap S = S \cap R$), associativa e idempotente ($R \cap R = R$), e seu resultado é sempre um subconjunto de ambas as relações originais.

Equivalência com Outras Operações

A interseção pode ser expressa em termos de diferença:

$R \cap S = R - (R - S)$, o que mostra que não é uma operação primitiva, mas é incluída por conveniência.

Operação Interseção – Exemplo Prático

Vejaamos como a interseção funciona identificando elementos comuns entre duas relações.

Relação R

Produto	Categoria
Notebook	Eletrônicos
Mouse	Periféricos
Teclado	Periféricos
Monitor	Eletrônicos

Produtos da Loja A

Relação S

Produto	Categoria
Mouse	Periféricos
Webcam	Periféricos
Notebook	Eletrônicos
Impressora	Periféricos

Produtos da Loja B

$R \cap S$

Produto	Categoria
Notebook	Eletrônicos
Mouse	Periféricos

Produtos em ambas as lojas

Análise do Resultado: Apenas Notebook e Mouse aparecem em ambas as relações R e S, portanto são os únicos produtos incluídos no resultado da interseção. Note que tanto "Teclado" e "Monitor" (exclusivos de R) quanto "Webcam" e "Impressora" (exclusivos de S) foram excluídos do resultado.

Operação de Diferença (-)

A operação de **diferença**, denotada pelo símbolo - ou \, produz uma relação contendo todas as tuplas que aparecem na primeira relação mas **não aparecem na segunda relação**.

Definição Formal

$R - S$ resulta em uma relação contendo todas as tuplas que estão em R mas não estão em S. As relações devem ser compatíveis em união.

Não-Comutatividade

A diferença **não é comutativa**: $R - S \neq S - R$. A ordem das operações é crucial e produz resultados diferentes.

Aplicações Práticas

Útil para encontrar registros exclusivos, como clientes inativos ou produtos descontinuados — qualquer cenário onde precisamos **excluir um conjunto de outro**.

❏ **Importante:** $R - S$ e $S - R$ são operações distintas e geralmente produzem resultados completamente diferentes. Sempre verifique a ordem correta para sua consulta.

Operação de Diferença – Exemplo Prático

Demonstraremos a diferença entre conjuntos e a importância da ordem na operação.

Relação R

ID	Nome
1	Alice
2	Bruno
3	Carlos
4	Diana

Clientes cadastrados

Relação S

ID	Nome
2	Bruno
4	Diana
5	Eduardo

Clientes com compras

$R - S$

ID	Nome
1	Alice
3	Carlos

Clientes sem compras

$S - R$

ID	Nome
5	Eduardo

Compras sem cadastro

Interpretação dos Resultados: $R - S$ identifica Alice e Carlos, que estão cadastrados mas nunca fizeram compras. Por outro lado, $S - R$ encontra Eduardo, que realizou uma compra mas não está no cadastro de clientes.

Operação de Produto Cartesiano ($\times$)

Conceito e Funcionamento

A operação de **produto cartesiano**, denotada por $\times$, combina duas relações criando uma nova relação com todas as combinações possíveis de tuplas das duas relações originais.

Se R tem n tuplas e m atributos, e S tem p tuplas e q atributos, então $R \times S$ terá $n \times p$ tuplas e $m + q$ atributos.

Notação e Características

- Não requer compatibilidade de união
- Combina cada tupla de R com cada tupla de S
- Pode gerar relações muito grandes
- Base para a operação de junção (join)

Número de Tuplas: $n \times p$

O resultado contém o produto do número de tuplas das duas relações

Número de Atributos: $m + q$

O resultado possui a soma dos atributos de ambas as relações

Operação de Produto Cartesiano – Exemplo Prático ($R \times S$)

Vejamos como o produto cartesiano combina todas as possíveis tuplas de duas relações.

Relação R

A	B
a1	b1
a2	b2

2 tuplas, 2 atributos

Relação S

C	D
c1	d1
c2	d2
c3	d3

3 tuplas, 2 atributos

$R \times S$

A	B	C	D
a1	b1	c1	d1
a1	b1	c2	d2
a1	b1	c3	d3
a2	b2	c1	d1
a2	b2	c2	d2
a2	b2	c3	d3

6 tuplas (2×3), 4 atributos ($2 + 2$)

📌 **Cuidado com o Tamanho:** O produto cartesiano pode crescer rapidamente. Se R tem 1.000 tuplas e S tem 1.000 tuplas, $R \times S$ terá 1.000.000 de tuplas! Por isso, é geralmente seguido por uma seleção para filtrar apenas as combinações relevantes, formando a base da operação de junção.

Expressão Algébrica: Definição Formal

Uma **expressão algébrica relacional** é construída pela composição de operações da álgebra relacional, permitindo expressar consultas complexas através da combinação estruturada de operações primitivas.

Operações Atômicas

Uma relação nomeada do banco de dados é uma expressão algébrica válida (caso base).

Composição Binária

Se E_1 e E_2 são expressões, então $E_1 \cup E_2$, $E_1 - E_2$ e $E_1 \times E_2$ também são expressões válidas.

Exemplo de Expressão Complexa

$\pi_{\text{nomeCliente}}(\sigma_{\text{cidadeCliente}=\text{'São Paulo'}}(\text{cliente}))$

Esta expressão encontra os nomes de todos os clientes que moram em São Paulo, primeiro selecionando (σ) os clientes dessa cidade na relação *cliente*, e depois projetando (π) apenas o atributo do nome.

Composição Unária

Se E é uma expressão, então $\sigma_P(E)$, $\pi_L(E)$ e $\rho_x(E)$ também são expressões válidas.

Precedência e Agrupamento

Parênteses controlam a ordem de avaliação das operações, de forma similar a expressões matemáticas.

Propriedades Importantes

- Fechamento:** O resultado de qualquer expressão algébrica é sempre uma relação
- Equivalência:** Múltiplas expressões podem produzir o mesmo resultado
- Otimização:** Expressões equivalentes podem ter custos de execução diferentes

Expressão Algébrica como Programa Relacional

Na álgebra relacional, uma expressão algébrica funciona como um programa completo de processamento de dados. Cada operador representa uma etapa específica de transformação, e o fluxo de execução é determinado pela estrutura hierárquica da expressão. Os resultados intermediários são relações temporárias que servem como entrada para as próximas operações, criando um pipeline de processamento de dados elegante e eficiente.

Exemplo de Expressão:

$\pi_{\text{nomeCliente}}(\sigma_{\text{cidadeCliente}=\text{'Rio de Janeiro'}}(\text{cliente}))$

Esta expressão seleciona todos os clientes do Rio de Janeiro (σ) e em seguida projeta (π) apenas seus nomes — demonstrando como as operações se compõem em um pipeline.

Exemplo de Expressão Algébrica

Uma expressão algébrica em bancos de dados relacionais combina operadores como seleção (σ), projeção (π), união ($\cup$), diferença ($-$) e produto cartesiano ($\times$) para manipular relações. Cada operação transforma uma ou mais relações de entrada em uma nova relação de saída.

A beleza da álgebra relacional está na sua capacidade de expressar consultas complexas através da composição de operações simples e bem definidas.

Exemplo Concreto:

$\pi_{\text{nomeCliente}}(\sigma_{\text{quantia} > 1200}(\text{tomador} \bowtie \text{emprestimo}))$

Esta expressão encontra os nomes dos clientes com empréstimos acima de \$1.200: faz a junção natural de tomador com emprestimo (pelo atributo numeroEmprestimo), seleciona (σ) os empréstimos acima desse valor e projeta (π) o nome do cliente. A junção é necessária porque emprestimo não possui nomeCliente.

 As expressões algébricas formam a base teórica para linguagens de consulta como SQL, permitindo otimizações automáticas pelo sistema gerenciador de banco de dados.

Equivalência de Expressões Algébricas

Mesmo Resultado

Expressões diferentes podem produzir resultados idênticos, mesmo quando estruturadas de formas distintas

Independência da Forma

A equivalência depende do resultado final, não da sintaxe ou ordem das operações

Otimização de Consultas

Permite ao SGBD escolher a execução mais eficiente entre alternativas equivalentes

Um exemplo clássico de equivalência ocorre com seleções e junções: aplicar uma seleção **antes** de uma junção pode ser equivalente a aplicá-la **depois**, mas a primeira abordagem geralmente é muito mais eficiente, pois reduz o número de tuplas processadas na junção.

Exemplos Concretos:

$$\sigma_{A=5}(\sigma_{B='X'}(R)) \equiv \sigma_{B='X'}(\sigma_{A=5}(R)) \equiv \sigma_{A=5 \wedge B='X'}(R)$$

Todas as três expressões filtram a relação R para tuplas onde A=5 e B='X', demonstrando a comutatividade das operações de seleção.

Operação de Atribuição

A operação de atribuição permite armazenar o resultado de uma expressão algébrica em uma relação temporária, facilitando a construção de consultas complexas em etapas.

Sintaxe básica: $\text{nome_relacao} \leftarrow \text{expressão_algébrica}$

Esta operação é fundamental para:

- Dividir consultas complexas em partes menores e mais compreensíveis
- Reutilizar resultados intermediários em múltiplas operações subsequentes
- Melhorar a legibilidade e manutenibilidade das expressões algébricas
- Facilitar o processo de otimização de consultas

Exemplos Práticos:

$$\begin{aligned}\text{temp1} &\leftarrow \sigma_{\text{nomeAgencia}='Perryridge'}(\text{emprestimo}) \\ \text{temp2} &\leftarrow \pi_{\text{numeroEmprestimo}, \text{quantia}}(\text{temp1}) \\ \text{resultado} &\leftarrow \text{temp2}\end{aligned}$$

Este exemplo divide uma consulta complexa em etapas: primeiro selecionando (σ) todos os empréstimos da agência Perryridge, depois projetando (π) apenas o número do empréstimo e a quantia.

Estrutura da Operação de Atribuição

Relação Original

Dados de entrada para processamento

Expressão Algébrica

Operações aplicadas aos dados

Símbolo $\leftarrow$

Operador de atribuição

Relação Temporária

Resultado armazenado

A atribuição permite construir consultas modulares, onde cada etapa representa uma transformação lógica dos dados. Por exemplo, podemos primeiro filtrar contas de uma agência específica, depois manter apenas aquelas com saldo elevado, e finalmente projetar apenas os campos desejados.

Exemplo de Pipeline:

```
contas_perryridge  $\leftarrow \sigma_{\text{nomeAgencia} = \text{'Perryridge'}}(\text{conta})$   
contas_altas  $\leftarrow \sigma_{\text{saldo} > 1000}(\text{contas\_perryridge})$   
resultado  $\leftarrow \pi_{\text{numeroConta}, \text{saldo}}(\text{contas\_altas})$ 
```

Este pipeline primeiro filtra as contas da agência Perryridge, depois mantém apenas aquelas com saldo acima de \$1.000, e finalmente projeta apenas o número da conta e o saldo.

Junção θ

A junção θ é uma operação derivada na álgebra relacional, o que significa que ela pode ser expressa e construída a partir de operações básicas como o produto cartesiano e a seleção.

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

Nesta definição formal, a condição θ (theta) é aplicada **após** o produto cartesiano das duas relações R e S , filtrando as tuplas que satisfazem a condição especificada. A junção natural, por exemplo, é um caso particular da junção θ , onde a condição θ é baseada na igualdade de atributos com o mesmo nome entre as relações envolvidas.

Exemplo de Junção θ :

$emprestimo \bowtie_{emprestimo.nomeAgencia=agencia.nomeAgencia} agencia$

Equivalente a:

$\sigma_{emprestimo.nomeAgencia=agencia.nomeAgencia}(emprestimo \times agencia)$

Este exemplo demonstra como a junção theta combina os empréstimos com os dados de suas respectivas agências, realizando a correspondência pelo atributo do nome da agência.

Operação Junção Natural (⋈)

A junção natural, denotada por $\bowtie$, é uma operação binária que combina duas relações com base na igualdade de valores em atributos de mesmo nome, eliminando colunas duplicadas no resultado. É uma operação derivada, definida formalmente como:

$$R \bowtie S = \pi_{\text{schema}}(\sigma_{\text{condição_igualdade}}(R \times S))$$

Identificação dos Atributos Comuns

Identificar todos os atributos que aparecem em ambas as relações R e S

Produto Cartesiano R × S

Calcular o produto cartesiano entre as duas relações, combinando todas as tuplas

Seleção por Igualdade

Selecionar apenas as tuplas onde os valores dos atributos comuns são iguais entre R e S

Projeção sem Duplicatas

Projetar o esquema final eliminando as colunas duplicadas dos atributos comuns

Propriedades

- Comutatividade:** $R \bowtie S = S \bowtie R$
- Associatividade:** $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$
- Caso especial:** Se R e S não têm atributos comuns, então $R \bowtie S = R \times S$

Exemplo com o Esquema Bancário

- depositante ⋈ tomador** — junção pelo atributo comum `nomeCliente`; retorna os clientes que são simultaneamente depositantes e tomadores de empréstimo:
`depositante(nomeCliente, numeroConta) ⋈ tomador(nomeCliente, numeroEmprestimo)`
- conta ⋈ agencia** — junção pelo atributo comum `nomeAgencia`; retorna os dados de cada conta enriquecidos com a cidade e o ativo da agência correspondente:
`conta(numeroConta, nomeAgencia, saldo) ⋈ agencia(nomeAgencia, cidadeAgencia, ativo)`

Junção Natural – Exemplo Prático

Demonstraremos a junção natural usando o esquema bancário. O atributo comum entre depositante e conta é numeroConta.

Expressão Algébrica:

$$\text{resultado} \leftarrow \text{depositante} \bowtie \text{conta}$$

Relação depositante

nomeCliente	numeroConta
Adams	A-102
Hayes	A-101
Johnson	A-102
Turner	A-305

Relação conta

numeroConta	nomeAgencia	saldo
A-101	Downtown	500
A-102	Perryridge	400
A-305	Round Hill	350

Resultado (depositante ⋈ conta)

nomeCliente	numeroConta	nomeAgencia	saldo
Adams	A-102	Perryridge	400
Hayes	A-101	Downtown	500
Johnson	A-102	Perryridge	400
Turner	A-305	Round Hill	350

O atributo numeroConta aparece apenas uma vez no resultado. Cada cliente é associado aos dados completos de sua conta. Observe que a junção natural é equivalente a:

$$\pi_{\text{nomeCliente}, \text{depositante.numeroConta}, \text{nomeAgencia}, \text{saldo}}(\sigma_{\text{depositante.numeroConta}=\text{conta.numeroConta}}(\text{depositante} \times \text{conta}))$$

No produto cartesiano, numeroConta aparece duas vezes; por isso é preciso qualificar o atributo na projeção.

❏ A junção natural é comutativa ($R \bowtie S = S \bowtie R$) e associativa ($(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$), propriedades fundamentais para otimização de consultas.

Projeção Generalizada

A projeção generalizada estende a operação de projeção tradicional, permitindo não apenas selecionar atributos existentes, mas também criar novos atributos através de expressões aritméticas e funções aplicadas aos atributos originais.

Seleção de Atributos

Escolha dos atributos existentes que farão parte do resultado, similar à projeção tradicional.

Cálculo de Expressões

Criação de novos atributos através de operações matemáticas: adição, subtração, multiplicação, divisão.

Aplicação de Funções

Uso de funções como CONCAT, SUBSTRING, UPPER e LOWER para manipular strings e outros tipos.

Renomeação

Atribuição de novos nomes aos atributos resultantes para maior clareza e usabilidade.

A seguir, alguns exemplos práticos que ilustram a criação de novos atributos através de expressões aritméticas:

$\pi_{\text{numeroConta}, \text{nomeAgencia}, \text{saldo}, \text{saldo} \times 0.05 \text{ as rendimento}}(\text{conta})$

Calcula um rendimento de 5% sobre o saldo de cada conta, criando um novo atributo `rendimento`.

$\pi_{\text{numeroEmprestimo}, \text{nomeAgencia}, \text{quantia}, \text{quantia} \times 1.10 \text{ as quantia_com_juros}}(\text{emprestimo})$

Acrescenta 10% de juros ao valor de cada empréstimo, criando um novo atributo `quantia_com_juros`.

Funções e Operações Agregadas

As operações agregadas permitem realizar cálculos sobre conjuntos de tuplas, produzindo valores resumidos que sintetizam características importantes dos dados. São essenciais para análises estatísticas e relatórios gerenciais.

SUM

Calcula a soma de valores numéricos em um atributo.

AVG

Determina a média aritmética dos valores.

COUNT

Conta o número de tuplas ou valores não nulos.

MAX

Identifica o valor máximo em um conjunto.

MIN

Identifica o valor mínimo em um conjunto.

Estas funções podem ser aplicadas sobre toda a relação ou sobre grupos de tuplas que compartilham valores em atributos específicos, utilizando a operação de agrupamento (GROUP BY).

Notação Formal

A forma geral da operação de agregação é:

$$G_1, \dots, G_n \Gamma_{F_1, \dots, F_m} (R)$$

onde $G_1, \dots, G_n$ são os atributos de agrupamento (escritos como subscrito à esquerda de Γ) e $F_1, \dots, F_m$ são as funções agregadas (escritas como subscrito de Γ). Quando não há agrupamento, os atributos G_i são omitidos e a função é aplicada sobre toda a relação.

Exemplos de Agregação

- $\Gamma_{\text{SUM(saldo)}}(\text{conta})$ - soma total dos saldos das contas
- $\Gamma_{\text{AVG(quantia)}}(\text{emprestimo})$ - média das quantias dos empréstimos
- $\Gamma_{\text{COUNT(nomeCliente)}}(\text{cliente})$ - contagem de clientes

Operação Agregada – Exemplo com SUM

Relação 'conta' Original

Considere uma relação 'conta' com um atributo numérico 'saldo' contendo os seguintes dados:

```
conta = {(500), (400), (350), (900)}
```

A operação agregada SUM(saldo) percorre todas as tuplas da relação 'conta', acumula os valores do atributo 'saldo' e retorna um único valor resultante.

Expressão e Resultado

A notação algébrica para a soma dos valores na relação 'conta' é:

$\Gamma_{\text{SUM(saldo)}}(\text{conta})$

Realizando o cálculo:

```
500 + 400 + 350 + 900 = 2150
```

SUM(saldo)
2150

O valor **2150** representa a soma de todos os valores presentes no atributo 'saldo' da relação 'conta'.

Este exemplo demonstra como uma operação agregada transforma múltiplas tuplas em um único valor resumido.

Operação Agregada com Agrupamento

Combinando agregação com agrupamento, calculamos estatísticas separadamente para cada grupo de tuplas que compartilham valores em atributos específicos.

Dados Originais

nomeAgencia	numeroConta	saldo
Perryridge	A-102	400
Perryridge	A-201	900
Brighton	A-217	750
Brighton	A-215	750
Redwood	A-222	700

Resultado Agregado por Agência

nomeAgencia	sum(saldo)
Perryridge	1300
Brighton	1500
Redwood	700

`nomeAgencia Γ SUM(saldo) (conta)`

A soma dos saldos é calculada separadamente para cada agência bancária.

Outro exemplo (COUNT e AVG):

Agrupa as contas por agência e calcula a quantidade de contas e a média dos saldos para cada agência.

`nomeAgencia Γ COUNT(numeroConta), AVG(saldo) (conta)`

Outro exemplo:

Agrupa os empréstimos por cliente e calcula a soma total das quantias emprestadas para cada um.

`nomeCliente Γ SUM(quantia) (tomador ⋈ emprestimo)`

Junção Externa (Outer Join)

A junção externa é uma extensão da operação de junção tradicional que evita a perda de informações quando algumas tuplas não encontram correspondência na outra relação.

Junção Tradicional

Mantém apenas tuplas com correspondência em ambas as relações.

Identificação de Perdas

Tuplas sem correspondência são descartadas, resultando em perda de informação.

Junção Externa

Preserva todas as tuplas, preenchendo com NULL onde não há correspondência.

Valores NULL representam informações desconhecidas, e comparações envolvendo NULL retornam um terceiro valor lógico: "desconhecido".

📌 Existem três tipos de junção externa: esquerda (LEFT), direita (RIGHT) e completa (FULL), cada uma preservando tuplas de diferentes lados da junção.

Exemplos de Junções Estendidas:

- **Left outer join:** depositante ⋈ conta (mantém todos os depositantes, mesmo sem conta correspondente)
- **Right outer join:** depositante ⋈ conta (mantém todas as contas, mesmo sem depositante correspondente)
- **Full outer join:** depositante ⋈ conta (mantém todos os depositantes e todas as contas)

Junção Interna (Inner Join)

A junção interna é a operação de junção tradicional, que mantém apenas as tuplas que possuem correspondência em ambas as relações. Tuplas sem correspondência são completamente eliminadas do resultado.

Expressão Algébrica:

Relação1 ⋈ Relação2 — junção natural pelo atributo comum b; equivale a Relação1 ⋈ Relação1.b = Relação2.b Relação2, removendo a coluna b duplicada.

Relação 1

a	b
L-170	Red
L-230	Green
L-160	Blue

Relação 2

b	c	d
Red	3000	Jones
Green	4000	Smith
Yellow	1000	Suzan

Resultado

a	b	c	d
L-170	Red	3000	Jones
L-230	Green	4000	Smith

Observe que a tupla com **b = Blue** da primeira relação e a tupla com **b = Yellow** da segunda relação não aparecem no resultado, pois não encontraram correspondência. A junção interna só inclui as tuplas que possuem correspondência em ambas as relações, como demonstrado no exemplo.

Junção Externa Esquerda (Left Outer Join)

A junção externa esquerda preserva **todas as tuplas da relação à esquerda**, mesmo aquelas que não encontram correspondência na relação à direita. Para tuplas sem correspondência, os atributos vindos da relação direita são preenchidos com NULL.

Expressão Algébrica:

`depositante |depositante.numeroConta=conta.numeroConta conta`

Resultado: Todos os depositantes aparecem. Depositantes sem conta correspondente terão NULL nos atributos de conta (nomeAgencia, saldo).

Relação depositante

nomeCliente	numeroConta
Adams	A-102
Hayes	A-101
Johnson	A-201

Relação conta

numero Conta	nome Agencia	saldo
A-101	Downtown	500
A-102	Perryridge	400
A-305	Round Hill	350

Resultado (depositante ⋈ conta)

nome Cliente	numero Conta	nome Agencia	saldo
Adams	A-102	Perryridge	400
Hayes	A-101	Downtown	500
Johnson	A-201	null	null

Johnson é preservado com NULL nos atributos de conta, pois não há conta A-201 correspondente. A conta A-305 (Round Hill) é descartada, pois esta é uma junção externa *esquerda*.

Junção Externa Direita (Right Outer Join)

A junção externa direita funciona de forma simétrica à esquerda: preserva **todas as tuplas da relação à direita**, mesmo aquelas sem correspondência na relação à esquerda. Atributos sem correspondência são preenchidos com NULL.

Expressão Algébrica:

depositante $\bowtie$ (depositante.numeroConta = conta.numeroConta) conta

Resultado: Todas as contas aparecem. Contas sem depositante correspondente terão NULL no atributo nomeCliente.

Relação depositante

nomeCliente	numeroConta
Adams	A-102
Hayes	A-101
Johnson	A-201

Relação conta

numero Conta	nome Agencia	saldo
A-101	Downtown	500
A-102	Perryridge	400
A-305	Round Hill	350

Resultado (depositante $\bowtie$ conta)

nome Cliente	numero Conta	nome Agencia	saldo
Adams	A-102	Perryridge	400
Hayes	A-101	Downtown	500
null	A-305	Round Hill	350

A conta **A-305 (Round Hill)** é preservada com NULL no atributo nomeCliente, pois não há depositante correspondente. **Johnson** (conta A-201) é descartado, pois esta é uma junção externa *direita*.

Junção Externa Integral (Full Outer Join)

A junção externa integral ou completa é a mais abrangente: preserva **todas as tuplas de ambas as relações**, independentemente de encontrarem correspondência ou não. É a união dos comportamentos das junções externas esquerda e direita.

Expressão Algébrica:

depositante $\bowtie$ (depositante.numeroConta = conta.numeroConta) conta

Resultado:

- Depositantes sem conta correspondente: NULL nos atributos de conta (nomeAgencia, saldo)
- Contas sem depositante correspondente: NULL no atributo nomeCliente
- Depositantes com conta correspondente: todos os atributos preenchidos

Esta operação demonstra a natureza abrangente da Junção Externa Integral, garantindo que nenhuma informação de ambas as relações seja perdida.

Relação depositante

nomeCliente	numeroConta
Adams	A-102
Hayes	A-101
Johnson	A-201

Relação conta

numeroConta	nomeAgencia	saldo
A-101	Downtown	500
A-102	Perryridge	400
A-305	Round Hill	350

Resultado Completo (depositante $\bowtie$ conta)

nome Cliente	numero Conta	nome Agencia	saldo
Adams	A-102	Perryridge	400
Hayes	A-101	Downtown	500
Johnson	A-201	null	null
null	A-305	Round Hill	350

Tanto **Johnson** (sem conta correspondente) quanto a conta **A-305 (Round Hill)** (sem depositante correspondente) são preservados. Esta é a forma mais completa de preservar dados em operações de junção.

Operação Divisão em Álgebra Relacional

A divisão é uma operação derivada da álgebra relacional, usada para expressar consultas com quantificação universal ("para todos"). Para $R(A, B)$ e $S(B)$: $R \div S = \pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$.

Expressão Algébrica:

$$R \div S$$

Exemplo Prático:

`estudante_curso(estudante, curso)  $\div$  cursos_obrigatorios(curso)`

Esta operação retorna os estudantes que cursaram TODOS os cursos obrigatórios, funcionando como a operação "para todos" na álgebra relacional.

Operação Divisão – Exemplo

Relações R e S

Expressão: $R \div S$

A operação divisão $R \div S$ retorna as tuplas de R que estão associadas a **todos** os valores em S. É como encontrar elementos que satisfazem uma condição "para todo".

Considere as relações ao lado: R contém pares (a, b) e S contém valores de b. O resultado $R \div S$ mostrará quais valores de 'a' aparecem com **todos** os valores de 'b' presentes em S, ou seja, para cada 'a' no resultado, deve haver uma tupla (a, b) em R para cada 'b' em S.

Tabelas de Exemplo

Relação R

a	b
α	1
α	2
β	1

Relação S

b
1
2

Resultado $R \div S$

a
α

No exemplo acima, a Relação S contém os valores {1, 2}. O único valor de 'a' na Relação R que está associado a **ambos** 1 e 2 é ' α '. Portanto, o resultado da divisão $R \div S$ é ' α '.

Outro Exemplo de Divisão

Relação R

c	d	e
w	x	1
w	y	1
z	x	1

Relação S

d	e
x	1
y	1

Resultado $R \div S$

c
w

Apenas 'w' aparece com todas as combinações (x,1) e (y,1) de S.

Expressão Algébrica: $R \div S$, onde R é R(c, d, e) e S é S(d, e).

O resultado da divisão $R \div S$ contém a coluna 'c' porque 'd' e 'e' são os atributos comuns a ambas as relações. A operação retorna o valor 'w' da relação R porque ele é o único valor de 'c' que está associado a **todas** as tuplas presentes em S, que são (x, 1) e (y, 1). Ou seja, existem as tuplas (w, x, 1) e (w, y, 1) em R.

Neste exemplo, a divisão identifica quais valores da coluna 'c' estão associados a todas as combinações de valores (d, e) presentes em S. A operação é útil para expressar consultas do tipo "encontre X que tem relacionamento com todos os Y".

Modificação do Banco de Dados

Além das operações de consulta, a álgebra relacional também permite modificar o conteúdo armazenado no banco de dados. Essas operações são essenciais para manter os dados atualizados e refletir as mudanças no mundo real.

Exclusão

Remove tuplas existentes de uma relação com base em uma condição específica.

$\text{conta} \leftarrow \text{conta} - \sigma_{\text{saldo} < 100}(\text{conta})$

Inserção

Adiciona novas tuplas a uma relação existente, individualmente ou como resultado de uma consulta.

$\text{cliente} \leftarrow \text{cliente} \cup \{('João', 'Rua das Flores', 'São Paulo')\}$

Atualização

Modifica valores de atributos em tuplas existentes sem remover as tuplas da relação.

$\text{conta} \leftarrow \pi_{\text{numeroConta}, \text{nomeAgencia}, \text{saldo} \times 1.05 \text{ as } \text{saldo}}(\text{conta})$

Todas essas operações são expressas usando o **operador de atribuição** ($\leftarrow$), que cria uma nova relação ou modifica uma existente com base no resultado de uma expressão de álgebra relacional.

Exclusão

Sintaxe Geral

$$r \leftarrow r - E$$

Onde:

- **r** é a relação
- **E** é uma expressão de álgebra relacional

Como Funciona

A operação de exclusão remove tuplas de uma relação usando a diferença de conjuntos, subtraindo as tuplas que satisfazem a condição especificada na expressão E.



Exemplos de Exclusão:

1. Para excluir todas as contas da agência 'Perryridge':

$$\text{conta} \leftarrow \text{conta} - \sigma_{\text{nomeAgencia}='Perryridge'}(\text{conta})$$

2. Para excluir todos os empréstimos com quantia inferior a 1000:

$$\text{emprestimo} \leftarrow \text{emprestimo} - \sigma_{\text{quantia} < 1000}(\text{emprestimo})$$

Inserção

Inserção de Tupla Única

$$r \leftarrow r \cup \{t\}$$

Adiciona uma tupla individual 't' à relação 'r' usando a operação de união. A tupla deve ter valores para todos os atributos da relação.

Inserção de Múltiplas Tuplas

$$r \leftarrow r \cup E$$

Insere o resultado de uma expressão de consulta 'E' na relação 'r'. Útil para copiar dados de outras relações ou inserir resultados de consultas complexas.

A operação de inserção utiliza a **união de conjuntos** para adicionar novas tuplas. É importante garantir que as tuplas inseridas tenham valores compatíveis com o esquema da relação, respeitando tipos de dados e restrições de integridade.

Expressões de Inserção:

$\text{conta} \leftarrow \text{conta} \cup \{('A-9732', 'Perryridge', 1200)\}$

$\text{conta} \leftarrow \text{conta} \cup \sigma_{\text{saldo} > 2000}(\text{conta_temporaria})$

Encontre Todos os Empréstimos de Mais de US\$ 1200

Encontre todos os registros de empréstimos cuja quantia seja superior a US\$ 1.200.

Solução

$$\sigma_{\text{quantia} > 1200}(\text{emprestimo})$$

A seleção filtra as tuplas da relação **emprestimo**, retornando apenas aquelas onde **quantia > 1.200**. Todos os atributos (numeroEmprestimo, nomeAgencia, quantia) são preservados no resultado.

Encontre o Número de Empréstimo para Quantias Maiores que US\$ 1200

Encontre o número de cada empréstimo cujo valor seja superior a US\$ 1.200.

Solução

$$\pi_{\text{numeroEmprestimo}}(\sigma_{\text{quantia} > 1200}(\text{emprestimo}))$$

A seleção filtra os empréstimos com quantia acima de 1.200. A projeção retém apenas o atributo numeroEmprestimo, descartando nomeAgencia e quantia.

Cientes com Empréstimo, Conta, ou Ambos

Encontre todos os clientes que possuem um empréstimo, uma conta, ou ambos no banco.

Solução

$$\pi_{\text{nomeCliente}}(\text{depositante}) \cup \pi_{\text{nomeCliente}}(\text{tomador})$$

Projetamos o nome dos clientes de cada relação separadamente e aplicamos a união. O resultado inclui cada cliente uma única vez, mesmo que apareça nas duas relações.

Cientes com Empréstimo E Conta no Banco

Encontre todos os clientes que possuem simultaneamente um empréstimo e uma conta no banco.

Solução

$$\pi_{\text{nomeCliente}}(\text{depositante}) \cap \pi_{\text{nomeCliente}}(\text{tomador})$$

A interseção retorna apenas os nomes que aparecem em ambas as relações — clientes que são ao mesmo tempo depositantes e tomadores de empréstimo.

Cientes com Empréstimo e Suas Quantias

Encontre o nome e a quantia de cada empréstimo, associando clientes aos seus respectivos empréstimos.

Solução

$$\pi_{\text{nomeCliente, quantia}}(\text{tomador} \bowtie \text{emprestimo})$$

A junção natural combina tomador e emprestimo pelo atributo comum numeroEmprestimo. A projeção seleciona apenas nomeCliente e quantia.

Calcular Crédito Disponível para Clientes

Dada a relação infoCredito(nomeCliente, limite, saldoCredito), calcule o crédito disponível de cada cliente.

Solução

$$\pi_{\text{nomeCliente}, \text{limite} - \text{saldoCredito}}(\text{infoCredito})$$

A projeção generalizada permite incluir expressões aritméticas. O atributo calculado limite – saldoCredito representa o valor ainda disponível para cada cliente.

Cientes com Contas em Todas as Agências de Brooklyn

Encontre todos os clientes que possuem conta em todas as agências localizadas em Brooklyn.

Solução

$$\text{temp1} \leftarrow \pi_{\text{nomeAgencia}}(\sigma_{\text{cidadeAgencia} = \text{'Brooklyn'}}(\text{agencia}))$$
$$\text{temp2} \leftarrow \pi_{\text{nomeCliente}, \text{nomeAgencia}}(\text{depositante} \bowtie \text{conta})$$
$$\text{resultado} \leftarrow \text{temp2} \div \text{temp1}$$

temp1 obtém todas as agências de Brooklyn. temp2 obtém os pares (cliente, agência) de cada depositante. A divisão retorna apenas os clientes associados a todas as agências de temp1.

Excluir Todas as Contas da Agência Perryridge

Remova da relação conta todos os registros pertencentes à agência Perryridge.

Solução

$$\text{conta} \leftarrow \text{conta} - \sigma_{\text{nomeAgencia} = \text{''Perryridge''}}(\text{conta})$$

A seleção identifica as tuplas da agência Perryridge. A diferença as remove da relação original. O resultado é atribuído de volta à relação conta.

Inserir Nova Conta no Banco de Dados

Insira na relação conta um novo registro para a conta A-973, da agência Perryridge, com saldo de US\$ 1.200. O cliente é Smith.

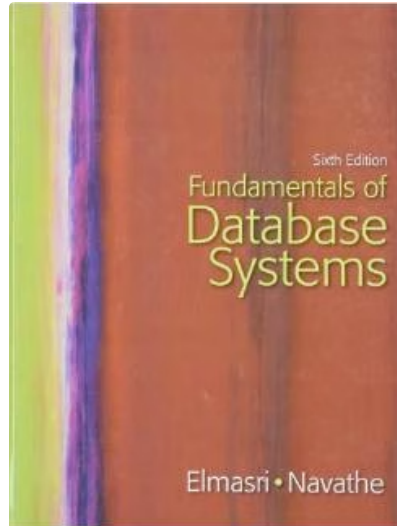
Solução

$$\text{conta} \leftarrow \text{conta} \cup \{(A-973, \text{Perryridge}, 1200)\}$$

$$\text{depositante} \leftarrow \text{depositante} \cup \{(\text{Smith}, A-973)\}$$

A inserção na relação conta usa a união com a nova tupla. Como o nome do cliente não pertence ao esquema de conta, ele é inserido separadamente na relação depositante.

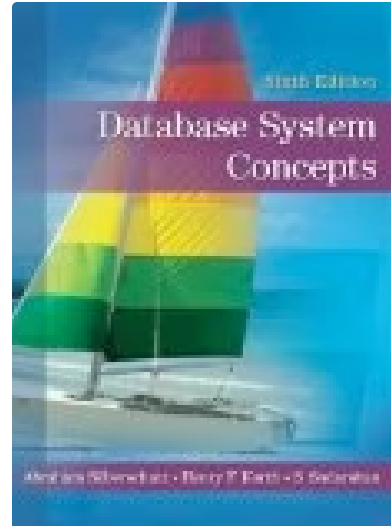
Referências



Elmasri & Navathe

Fundamentals of Database Systems

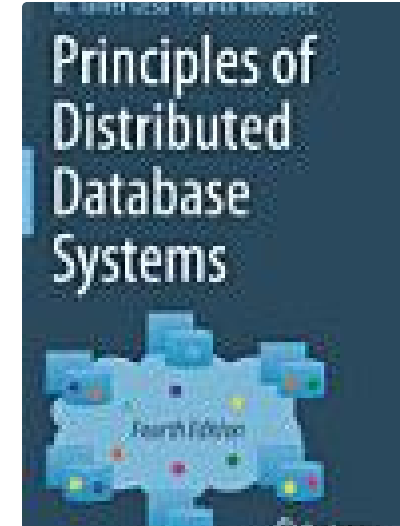
Pearson, 2016



Korth, Sudarshan & Silberschatz

Database System Concepts

McGraw-Hill, 2019



Özsu & Valduriez

Principles of Distributed Database Systems

Springer Nature, 2019